

Sampling and Monte Carlo Methods

Table of Contents

1. [Introduction](#)
 2. [What is Sampling?](#)
 3. [Sampling from Discrete Distributions](#)
 4. [Sampling from Continuous Distributions](#)
 5. [Monte Carlo Estimation](#)
 6. [Reconstructing Distributions from Samples](#)
 7. [Practical Applications](#)
 8. [Implementation in NumPy](#)
-

Introduction

Sampling is the process of drawing random values from a probability distribution. **Monte Carlo methods** use random sampling to solve problems that might be deterministic in principle.

Why sampling?

- Simulate random processes
 - Approximate complex calculations
 - Estimate expectations when analytical solutions are hard
 - Generate synthetic data
 - Test algorithms
-

What is Sampling?

Definition

Sampling means generating random values that follow a particular probability distribution.

Example:

- Distribution: Fair coin ($P(\text{Heads})=0.5$, $P(\text{Tails})=0.5$)
- Sampling: Flip the coin and record the outcome
- Many samples: {H, T, H, H, T, H, T, T, ...}

Properties of Good Samples

If we draw many samples from a distribution:

1. **Frequency matches probability:** $P(X=x) \approx (\text{count of } x) / (\text{total samples})$
2. **Sample mean $\rightarrow$ Expected value:** $\text{mean}(\text{samples}) \rightarrow E$ ☒ as $n \rightarrow \infty$
3. **Sample variance $\rightarrow$ True variance:** $\text{var}(\text{samples}) \rightarrow \text{Var}$ ☒ as $n \rightarrow \infty$

This is the **Law of Large Numbers!**

Sampling from Discrete Distributions

Method 1: Using np.random.choice

```
import numpy as np

# Define PMF
outcomes = [0, 1, 2, 3]
probabilities = [0.1, 0.3, 0.4, 0.2]

# Draw 1000 samples
samples = np.random.choice(outcomes, size=1000, p=probabilities)

print("Sample mean:", np.mean(samples))
print("Expected value:", np.sum(outcomes * probabilities))
```

Method 2: Sampling from 2D Grid PMF

For a 2D PMF like `submarine_pmf`, we need to:

1. Flatten the 2D PMF to 1D
2. Create corresponding coordinate list
3. Sample indices
4. Convert indices back to coordinates

```

def sample_from_2d_pmf(pmf, n_samples):
    """
    Sample from a 2D probability distribution

    Parameters:
    -----
    pmf : array (n_x, n_y)
        2D probability mass function
    n_samples : int
        Number of samples to draw

    Returns:
    -----
    samples : array (n_samples, 2)
        Sampled coordinates [x, y]
    """
    # Flatten PMF
    probabilities = pmf.ravel()

    # Create list of all possible coordinates
    n_x, n_y = pmf.shape
    coords = [[x, y] for x in range(n_x) for y in range(n_y)]

    # Sample indices
    indices = np.arange(n_x * n_y)
    sampled_indices = np.random.choice(indices, size=n_samples, p=probabilities)

    # Convert to coordinates
    samples = np.array([coords[i] for i in sampled_indices])

    return samples

# Example
samples = sample_from_2d_pmf(submarine_pmf, 1000)
print(f"Shape: {samples.shape}") # (1000, 2)
print(f"First 5 samples:\n{samples[:5]}")

```

Method 3: More Efficient 2D Sampling

```
def sample_from_2d_pmf_efficient(pmf, n_samples):  
    """Efficient sampling from 2D PMF"""  
    # Flatten and sample  
    flat_samples = np.random.choice(  
        np.arange(pmf.size),  
        size=n_samples,  
        p=pmf.ravel()  
    )  
  
    # Convert flat indices to 2D coordinates  
    # unravel_index converts flat index to (row, col)  
    samples = np.array(np.unravel_index(flat_samples, pmf.shape)).T  
  
    return samples  
  
# Example  
samples = sample_from_2d_pmf_efficient(submarine_pmf, 1000)
```

Verification

```
# Verify samples follow the distribution  
n_samples = 10000  
samples = sample_from_2d_pmf(submarine_pmf, n_samples)  
  
# Empirical mean should be close to theoretical mean  
xx, yy = np.meshgrid(np.arange(16), np.arange(16))  
expected_x = np.sum(xx.T * submarine_pmf)  
expected_y = np.sum(yy.T * submarine_pmf)  
  
sample_mean_x = np.mean(samples[:, 0])  
sample_mean_y = np.mean(samples[:, 1])  
  
print(f"Expected location: [{expected_x:.2f}, {expected_y:.2f}]")  
print(f"Sample mean location: [{sample_mean_x:.2f}, {sample_mean_y:.2f}]")
```

Sampling from Continuous Distributions

Common Continuous Distributions in NumPy

```
# Normal distribution
samples = np.random.normal(loc=0, scale=1, size=1000) #  $\mu=0$ ,  $\sigma=1$ 

# Uniform distribution
samples = np.random.uniform(low=0, high=1, size=1000) # [0, 1]

# Exponential distribution
samples = np.random.exponential(scale=1.0, size=1000) #  $\lambda=1$ 

# Multivariate normal
mean = [0, 0]
cov = [[1, 0.5], [0.5, 1]]
samples = np.random.multivariate_normal(mean, cov, size=1000)
```

Using scipy.stats

```
import scipy.stats

# Create distribution object
dist = scipy.stats.norm(loc=0, scale=1)

# Sample
samples = dist.rvs(size=1000)

# Can also compute PDF, CDF, etc.
pdf_values = dist.pdf(samples)
```

Multivariate Normal Distribution

```
import numpy as np
from scipy.stats import multivariate_normal

# Parameters
mu = np.array([6, 6])
sigma = np.array([[1.0, 0.6],
                  [0.6, 1.0]])

# Create distribution
mvn = multivariate_normal(mean=mu, cov=sigma)

# Sample
samples = mvn.rvs(size=1000) # Shape: (1000, 2)

# Verify
print("Expected mean:", mu)
print("Sample mean:", np.mean(samples, axis=0))
print("\nExpected covariance:\n", sigma)
print("Sample covariance:\n", np.cov(samples.T))
```

Monte Carlo Estimation

Monte Carlo estimation uses random sampling to approximate quantities that are hard to compute analytically.

Key Idea

To estimate $E[f(X)]$:

1. Draw n samples: $x_1, x_2, \dots, x_n \sim P(X)$
2. Compute $f(x_i)$ for each sample
3. Estimate: $E[f(X)] \approx (1/n) \sum f(x_i)$

By the Law of Large Numbers: This estimator converges to the true expectation as $n \rightarrow \infty$

Example 1: Estimating π

```
#  $\pi$  can be estimated by sampling random points in a square

n_samples = 1000000

# Sample random points in  $[-1, 1] \times [-1, 1]$ 
x = np.random.uniform(-1, 1, n_samples)
y = np.random.uniform(-1, 1, n_samples)

# Count how many fall inside unit circle
inside_circle = (x**2 + y**2) <= 1
fraction_inside = np.sum(inside_circle) / n_samples

# Area of circle / Area of square =  $\pi/4$ 
pi_estimate = 4 * fraction_inside

print(f"Estimated  $\pi$ : {pi_estimate:.4f}")
print(f"True  $\pi$ : {np.pi:.4f}")
print(f"Error: {abs(pi_estimate - np.pi):.4f}")
```

Example 2: Expected Distance

Problem: Estimate the expected distance from submarine to station at $[2, 5]$.

Analytical approach (hard):

```
# Compute distance at each grid point
station = np.array([2, 5])
xx, yy = np.meshgrid(np.arange(16), np.arange(16))
distances = np.sqrt((xx.T - station[0])**2 + (yy.T - station[1])**2)

# Expected value
expected_distance = np.sum(distances * submarine_pmf)
```


Monte Carlo approach (easy!):

```
# Sample submarine locations
n_samples = 10000
samples = sample_from_2d_pmf(submarine_pmf, n_samples)

# Compute distance for each sample
station = np.array([2, 5])
distances = np.sqrt(np.sum((samples - station)**2, axis=1))

# Estimate expected distance
expected_distance_mc = np.mean(distances)

print(f"Analytical: {expected_distance:.3f}")
print(f"Monte Carlo: {expected_distance_mc:.3f}")
```

Example 3: Expected Search Time

Problem: Search time is $t = d^2 + 3d + 2$. What's the expected time?

```
def search_time(distance):  
    """Compute search time given distance"""  
    return distance**2 + 3*distance + 2  
  
# Monte Carlo estimation  
n_samples = 10000  
samples = sample_from_2d_pmf(submarine_pmf, n_samples)  
  
# Compute distance from station [2, 5]  
station = np.array([2, 5])  
distances = np.sqrt(np.sum((samples - station)**2, axis=1))  
  
# Compute time for each sample  
times = search_time(distances)  
  
# Expected time  
expected_time = np.mean(times)  
std_error = np.std(times) / np.sqrt(n_samples)  
  
print(f"Expected search time: {expected_time:.2f} ± {std_error:.2f} hours")
```

Example 4: Probability Estimation

Problem: What's the probability the submarine is in the box [2, 2] to [6, 6]?

Analytical:

```
prob_analytical = np.sum(submarine_pmf[2:7, 2:7])
```

Monte Carlo:

```
n_samples = 10000
samples = sample_from_2d_pmf(submarine_pmf, n_samples)

# Count samples in box
in_box = ((samples[:, 0] >= 2) & (samples[:, 0] <= 6) &
          (samples[:, 1] >= 2) & (samples[:, 1] <= 6))
prob_mc = np.sum(in_box) / n_samples

print(f"Analytical: {prob_analytical:.4f}")
print(f"Monte Carlo: {prob_mc:.4f}")
```

Convergence and Error

The **standard error** of a Monte Carlo estimate decreases as $\sqrt{n}$:

$$\text{Standard Error} = \sigma / \sqrt{n}$$

where σ is the standard deviation of $f(X)$.

To reduce error by factor of 10, need 100× more samples!

```
# Demonstrate convergence
sample_sizes = [10, 100, 1000, 10000, 100000]
station = np.array([2, 5])

# True expected distance (analytical)
xx, yy = np.meshgrid(np.arange(16), np.arange(16))
distances = np.sqrt((xx.T - station[0])**2 + (yy.T - station[1])**2)
true_expected = np.sum(distances * submarine_pmf)

for n in sample_sizes:
    samples = sample_from_2d_pmf(submarine_pmf, n)
    dists = np.sqrt(np.sum((samples - station)**2, axis=1))
    estimate = np.mean(dists)
    error = abs(estimate - true_expected)
    std_error = np.std(dists) / np.sqrt(n)

    print(f"n={n:6d}: estimate={estimate:.4f}, "
          f"error={error:.4f}, SE={std_error:.4f}")
```

Reconstructing Distributions from Samples

Given samples from a distribution, we can reconstruct an approximation of the original distribution.

For Discrete Distributions

Method: Count frequency of each outcome

```

def reconstruct_pmf_from_samples(samples, shape=(16, 16)):
    """
    Reconstruct 2D PMF from samples

    Parameters:
    -----
    samples : array (n_samples, 2)
        Sampled coordinates
    shape : tuple
        Shape of the PMF

    Returns:
    -----
    pmf_reconstructed : array (shape)
        Reconstructed PMF (empirical distribution)
    """
    n_samples = samples.shape[0]
    pmf = np.zeros(shape)

    # Count frequency of each coordinate
    for sample in samples:
        x, y = int(sample[0]), int(sample[1])
        pmf[x, y] += 1

    # Normalize to get probabilities
    pmf = pmf / n_samples

    return pmf

# Example
n_samples = 10000
samples = sample_from_2d_pmf(submarine_pmf, n_samples)
reconstructed_pmf = reconstruct_pmf_from_samples(samples)

# Compare
print("Original PMF at [7, 10]:", submarine_pmf[7, 10])
print("Reconstructed PMF at [7, 10]:", reconstructed_pmf[7, 10])

# Visualize

```

```
show_pmf(submarine_pmf, "Original")
show_pmf(reconstructed_pmf, "Reconstructed from 10,000 samples")
```

More Efficient Reconstruction

```
def reconstruct_pmf_efficient(samples, shape=(16, 16)):
    """More efficient reconstruction using np.unique"""
    # Count occurrences of each coordinate
    # Convert to tuple for use with np.unique
    coords_as_tuples = [tuple(s) for s in samples]

    # Count frequencies
    unique_coords, counts = np.unique(coords_as_tuples,
                                       return_counts=True,
                                       axis=0)

    # Create PMF
    pmf = np.zeros(shape)
    for coord, count in zip(unique_coords, counts):
        pmf[int(coord[0]), int(coord[1])] = count

    # Normalize
    pmf = pmf / len(samples)

    return pmf
```

Using np.histogram2d

```
def reconstruct_pmf_histogram(samples, shape=(16, 16)):
    """Reconstruct using np.histogram2d"""
    # Create 2D histogram
    H, xedges, yedges = np.histogram2d(
        samples[:, 0],
        samples[:, 1],
        bins=[np.arange(shape[0] + 1), np.arange(shape[1] + 1)]
    )

    # Normalize
    pmf = H / np.sum(H)

    return pmf

# Example
samples = sample_from_2d_pmf(submarine_pmf, 10000)
reconstructed = reconstruct_pmf_histogram(samples)
```

For Continuous Distributions

For continuous distributions, use **kernel density estimation (KDE)**:

```
from scipy.stats import gaussian_kde

# Sample from distribution
true_samples = np.random.normal(loc=5, scale=2, size=1000)

# Fit KDE
kde = gaussian_kde(true_samples)

# Evaluate at points
x_eval = np.linspace(0, 10, 100)
pdf_estimate = kde(x_eval)

# Compare to true PDF
true_pdf = scipy.stats.norm(loc=5, scale=2).pdf(x_eval)

import matplotlib.pyplot as plt
plt.plot(x_eval, true_pdf, 'r-', label='True PDF')
plt.plot(x_eval, pdf_estimate, 'b--', label='KDE Estimate')
plt.hist(true_samples, bins=30, density=True, alpha=0.3, label='Histogram')
plt.legend()
plt.show()
```

Applications

Application 1: Simulating Random Processes

Problem: Simulate a submarine randomly drifting over time.


```
# Initial distribution
current_position = sample_from_2d_pmf(submarine_pmf, 1)

# Simulate drift over 10 time steps
positions = [current_position[0]]

for t in range(10):
    # Drift: add random noise
    drift = np.random.normal(0, 0.5, 2)
    current_position = current_position + drift

    # Clip to grid
    current_position = np.clip(current_position, 0, 15)

    positions.append(current_position.copy())

positions = np.array(positions)

# Visualize trajectory
plt.plot(positions[:, 0], positions[:, 1], 'o-')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Simulated submarine drift')
plt.show()
```

Application 2: Uncertainty Quantification

Problem: How uncertain are we about the expected distance?

```
# Bootstrap: resample and recompute
n_bootstrap = 1000
n_samples = 1000
station = np.array([2, 5])

estimates = []
for _ in range(n_bootstrap):
    # Sample from distribution
    samples = sample_from_2d_pmf(submarine_pmf, n_samples)

    # Compute expected distance
    dists = np.sqrt(np.sum((samples - station)**2, axis=1))
    estimates.append(np.mean(dists))

estimates = np.array(estimates)

# 95% confidence interval
ci_lower = np.percentile(estimates, 2.5)
ci_upper = np.percentile(estimates, 97.5)

print(f"Expected distance: {np.mean(estimates):.3f}")
print(f"95% CI: [{ci_lower:.3f}, {ci_upper:.3f}]" )
```

Application 3: Integration

Monte Carlo can estimate integrals:

$$\int f(x) dx \approx (b - a) / n \cdot \sum f(x_i) \quad \text{where } x_i \sim \text{Uniform}(a, b)$$

Example: Estimate $\int_0^1 x^2 dx$

```
n_samples = 10000
x = np.random.uniform(0, 1, n_samples)
f_x = x**2

integral_estimate = (1 - 0) * np.mean(f_x)
integral_true = 1/3

print(f"Estimate: {integral_estimate:.4f}")
print(f"True value: {integral_true:.4f}")
print(f"Error: {abs(integral_estimate - integral_true):.4f}")
```

Implementation in NumPy

Template: Generic Sampling Function

```
def sample_discrete(outcomes, probabilities, n_samples):  
    """  
    Sample from a discrete distribution  
  
    Parameters:  
    -----  
    outcomes : array-like  
        Possible outcomes  
    probabilities : array-like  
        Probability of each outcome  
    n_samples : int  
        Number of samples  
  
    Returns:  
    -----  
    samples : array  
        Sampled outcomes  
    """  
    return np.random.choice(outcomes, size=n_samples, p=probabilities)
```

Template: Monte Carlo Expectation

```
def monte_carlo_expectation(distribution, function, n_samples):
    """
    Estimate  $E[f(X)]$  using Monte Carlo

    Parameters:
    -----
    distribution : callable
        Function that returns samples: distribution(n) -> array (n, ...)
    function : callable
        Function to compute expectation of: f(x) -> scalar or array
    n_samples : int
        Number of Monte Carlo samples

    Returns:
    -----
    estimate : float or array
        Monte Carlo estimate of  $E[f(X)]$ 
    std_error : float or array
        Standard error of estimate
    """
    # Draw samples
    samples = distribution(n_samples)

    # Compute function values
    values = np.array([function(x) for x in samples])

    # Estimate expectation
    estimate = np.mean(values, axis=0)
    std_error = np.std(values, axis=0) / np.sqrt(n_samples)

    return estimate, std_error

# Example usage
def sample_submarine(n):
    return sample_from_2d_pmf(submarine_pmf, n)

def distance_from_station(location):
    station = np.array([2, 5])
```

```
        return np.sqrt(np.sum((location - station)**2))

estimate, std_error = monte_carlo_expectation(
    sample_submarine,
    distance_from_station,
    n_samples=10000
)

print(f"Expected distance: {estimate:.3f} ± {std_error:.3f}")
```

Common Mistakes

Mistake 1: Not setting random seed

```
# Without seed: results change every run
samples1 = sample_from_2d_pmf(submarine_pmf, 100)
samples2 = sample_from_2d_pmf(submarine_pmf, 100)
assert not np.array_equal(samples1, samples2) # Different!

# With seed: reproducible
np.random.seed(42)
samples1 = sample_from_2d_pmf(submarine_pmf, 100)
np.random.seed(42)
samples2 = sample_from_2d_pmf(submarine_pmf, 100)
assert np.array_equal(samples1, samples2) # Same!
```

Mistake 2: Confusing sample size with accuracy

```
# These have same accuracy:
np.random.seed(42)
small = sample_from_2d_pmf(submarine_pmf, 100)

np.random.seed(42)
large = sample_from_2d_pmf(submarine_pmf, 100)

# These have different accuracy:
np.random.seed(42)
est_small = np.mean([sample_from_2d_pmf(submarine_pmf, 100)
                     for _ in range(1)])

np.random.seed(42)
est_large = np.mean([sample_from_2d_pmf(submarine_pmf, 10000)
                     for _ in range(1)])
```

Mistake 3: Forgetting to normalize reconstructed PMF

```
# WRONG
pmf = np.zeros((16, 16))
for sample in samples:
    pmf[int(sample[0]), int(sample[1])] += 1
# This contains counts, not probabilities!

# CORRECT
pmf = pmf / len(samples) # Normalize
```

Mistake 4: Using wrong axis

```
# Wrong axis
samples = np.random.multivariate_normal([0, 0], [[1, 0], [0, 1]], 1000)
mean_wrong = np.mean(samples, axis=1) # Shape: (1000,) - Wrong!

# Correct
mean_correct = np.mean(samples, axis=0) # Shape: (2,) - Correct!
```

Summary

Sampling:

- Drawing random values from a probability distribution
- Foundation for Monte Carlo methods and simulation

Monte Carlo Estimation:

$$E[f(X)] \approx (1/n) \sum f(x_i) \quad \text{where } x_i \sim P(X)$$

Key Concepts:

1. **Law of Large Numbers:** Sample mean $\rightarrow$ True mean as $n \rightarrow \infty$
2. **Standard Error:** $SE = \sigma/\sqrt{n}$ (decreases slowly!)
3. **Convergence:** Need 100x more samples to reduce error by 10x

NumPy Functions:

- `np.random.choice()` : Sample from discrete distribution
- `np.random.normal()` : Sample from normal distribution
- `np.random.multivariate_normal()` : Sample from multivariate normal
- `scipy.stats.rv.rvs()` : Sample from any scipy distribution

Applications:

- Computing expectations analytically is hard
- Simulating random processes
- Approximating integrals
- Uncertainty quantification
- Generating synthetic data

Best Practices:

- Set random seed for reproducibility
- Use enough samples (check convergence!)
- Compute standard errors to quantify uncertainty
- Verify with analytical solutions when possible
- Vectorize operations for speed